

Iterative Improvement and Variable Neighborhood Descent for the Linear Ordering Problem

Rahman Tektas

542557

ULB

rahman.tektas@ulb.be

ABSTRACT

This report studies local search algorithms for the Linear Ordering Problem (LOP). First, iterative improvement algorithms were implemented using two pivoting rules, first-improvement and best-improvement, combined with the three neighborhoods transpose, exchange, and insert. Two initialization methods were considered: a random permutation and the Chenery–Watanabe (CW) heuristic. This yields twelve algorithmic combinations. Second, two Variable Neighborhood Descent (VND) variants were implemented, using the neighborhood orders transpose–exchange–insert and transpose–insert–exchange, with CW initialization and first-improvement only. The algorithms were evaluated on the benchmark instances provided for the course using the average percentage deviation from the best known solutions and the total computation time. Statistical comparisons were carried out with paired Student t-tests and Wilcoxon signed-rank tests in R. The results show that the insert neighborhood clearly outperforms exchange and transpose in terms of solution quality, that the effect of initialization is strong for weak neighborhoods such as transpose but much smaller for insert, and that the two VND variants do not differ significantly in solution quality, although transpose–insert–exchange is slightly better on average and faster overall.

1 INTRODUCTION

The Linear Ordering Problem (LOP) is a combinatorial optimization problem defined on a square matrix C . The goal is to find a permutation of the rows and columns that maximizes the sum of matrix entries above the diagonal in the reordered matrix. Because the problem is hard, heuristic methods are often used to obtain good solutions in reasonable time.

In this work, two families of local search methods are studied. In Exercise 1.1, iterative improvement algorithms are compared under different pivoting rules, neighborhoods, and initialization methods. In Exercise 1.2, Variable Neighborhood Descent (VND) is used to combine several neighborhoods within one algorithm. The objective is to study how these design choices affect solution quality and runtime.

2 PROBLEM DESCRIPTION

Let C be an $n \times n$ matrix, where c_{ij} denotes the entry in row i and column j . A solution of the LOP is a permutation π of the indices $\{1, \dots, n\}$. The objective function is

$$f(\pi) = \sum_{i=1}^n \sum_{j=i+1}^n c_{\pi(i)\pi(j)}.$$

The goal is to find a permutation π that maximizes $f(\pi)$. Three neighborhoods were considered in this work:

- **Transpose:** swap two adjacent elements.
- **Exchange:** swap two elements at arbitrary positions.
- **Insert:** remove one element from a position and insert it at another position.

Transpose is the smallest neighborhood, while exchange and insert allow larger changes to the ordering.

3 MATERIALS AND METHODS

3.1 Implementation and Reproducibility

The algorithms were implemented in C and compiled on Linux using a Makefile, which produces the executable `lop`. A single executable was used for all algorithmic variants, and the desired method was selected through command-line parameters. The source code is organized in the `src/` directory, and the Makefile is located at the root of the project. The results were saved in the `results/` directory, which contains the raw output files generated from running the experiments.

Additional scripts were used to automate the experiments and generate the raw result files, inside the `scripts/` folder. The main raw outputs were saved in `exercise1_results.txt` and `exercise2_vnd_results.txt`. The statistical analyses were then carried out in R language.

A README file is included in the submission and contains the exact compilation and execution commands required to reproduce all experiments from the command line.

3.2 Iterative Improvement Algorithms

For exercise 1.1, iterative improvement algorithms were implemented using :

- two pivoting rules:
 - first-improvement
 - best-improvement
- three neighborhoods:
 - transpose
 - exchange
 - insert
- two initialization methods:
 - random permutation
 - Chenery–Watanabe (CW) heuristic

This yields $2 \times 3 \times 2 = 12$ combinations.

3.3 Incremental update

To avoid unnecessary computations, incremental updates of the evaluation function value were implemented for each neighborhood. This makes it possible to evaluate the objective function right after a move, avoiding the recomputation of the full objective function from scratch.

Let

$$\pi = (\pi_1, \pi_2, \dots, \pi_n)$$

be the current permutation, and let

$$f(\pi) = \sum_{h=1}^n \sum_{k=h+1}^n c_{\pi_h \pi_k}.$$

Denote by $\Delta = f(\pi') - f(\pi)$ the change in objective value after applying a move.

- **Transpose:** swapping adjacent elements at indices i and $i+1$:

$$\Delta = c_{\pi_{i+1} \pi_i} - c_{\pi_i \pi_{i+1}}.$$

- **Exchange:** swapping elements at indices i and j , with $i < j$:

$$\Delta = c_{\pi_j \pi_i} - c_{\pi_i \pi_j} + \sum_{k=i+1}^{j-1} (c_{\pi_k \pi_i} - c_{\pi_i \pi_k} + c_{\pi_j \pi_k} - c_{\pi_k \pi_j}).$$

- **Insert:** removing the element at index i and inserting it at index j :
if $i < j$,

$$\Delta = \sum_{k=i+1}^j (c_{\pi_k \pi_i} - c_{\pi_i \pi_k}),$$

and if $i > j$,

$$\Delta = \sum_{k=j}^{i-1} (c_{\pi_i \pi_k} - c_{\pi_k \pi_i}).$$

3.4 Chenery–Watanabe Heuristic

This heuristic constructs the solution each row at a time, at each step selecting the most attractive remaining row, which is the one that maximizes the sum of elements having an influence on objective function.

The attractiveness of a row i at construction step s can be defined as such :

$$\sum_{j=s+1}^n c_{\pi(i)j}.$$

This means the heuristic prefers the rows that have the highest remaining contribution to the objective function, so this method is intended to provide better initial solutions than an uniformed random starting point.

3.5 Variable Neighborhood Descent

For exercise 1.2, 2 Variable Neighborhood Descent (VND) variants were implemented, using the neighborhood orders:

- (1) transpose $\rightarrow$ exchange $\rightarrow$ insert
- (2) transpose $\rightarrow$ insert $\rightarrow$ exchange

In both cases, only the first-improvement pivoting rule was used, and the CW heuristic was used for initialization.

The VND algorithm starts from the first neighborhood, and if an improving move is found, it applies it and restarts from the first neighborhood. If no improving move exists it just moves to the next neighborhood, and so on. The algorithm terminates when no improving move exists in any neighborhood.

4 RESULTS

The benchmark instances used in the experiments were the provided LOP instances of sizes 150 and 250. Each algorithm was applied once to each instance, and a random seed was set for reproducibility and fair comparison. For each run, two values were recorded:

- the final solution cost
- the computation time

Below is a summary table, where solution quality is measured using the relative percentage deviation from the best known solution, defined as

$$\Delta_{ki} = 100 \cdot \frac{\text{best-known}_i - \text{cost}_{ki}}{\text{best-known}_i}.$$

With this convention, smaller values correspond to better solutions.

4.1 Exercise 1.1 : Summary of the 12 Iterative Improvement Algorithms

Table 1: Summary results for the 12 iterative improvement algorithms.

Algorithm Variant	Avg. Dev. (%)	Std. Dev.	Time (s)
cw best exchange	3.8465	0.6054	32.9845
cw best insert	2.2608	0.4065	33.7210
cw best transpose	17.0783	1.6065	0.2194
cw first exchange	2.9660	0.4613	186.7398
cw first insert	2.0164	0.4109	128.2090
cw first transpose	17.0863	1.6042	0.2148
random best exchange	3.6027	0.5048	35.8138
random best insert	2.3037	0.4635	37.9277
random best transpose	34.4701	3.8033	0.0035
random first exchange	2.8159	0.4810	228.0147
random first insert	2.0211	0.4507	189.8870
random first transpose	34.6058	3.7984	0.0024

Table 1 highlights several trends. First, the insert neighborhood gives the best overall solution quality, since the average deviations are the lowest among the three neighborhoods. The exchange neighborhood performs a bit worse than insert, while the transpose neighborhood performs significantly worse than both insert and exchange. Second, the effect of initialization method depends strongly on the neighborhood. For transpose, the CW heuristic gives much better results than random initialization, the CW-based transpose variants achieve deviations around 17%, whereas the random transpose variants remain around 34%. while for exchange and insert, the difference is smaller and less consistent. So this suggests

that when the neighborhood is weak, the choice of initialization method has a strong impact on the solution quality. In contrast the deviation values for the exchange and insert neighborhoods are much closer, which suggests that the neighborhood is strong enough to compensate the poor initial solution.

Third, the comparison of pivoting rules shows that best-improvement does not always give better results than first-improvement. For the insert neighborhood, the first-improvement achieve slightly better results than best-improvement, both for random and CW initialization. Same thing happens for the exchange neighborhood, with first-improvement again producing smaller deviations than best-improvement.

Finally, in terms of computation time, the transpose neighborhood is by far the fastest, since it only involves swapping adjacent elements and has a small neighborhood size. The exchange and insert neighborhoods are more computationally expensive, but the speed comes with poor solution quality. Exchange and insert have higher computation times, especially for the first-improvement variants, but do yield better results.

Between all the neighborhoods, insert offers the best compromise between solution quality and computation time, which makes it the most effective choice in these experiments.

Table 2: Summary results for the two VND algorithms.

Method	Avg. Deviation (%)	Std. Dev.	Total Time (s)
vnd-tei	-2.0633	0.4191	109.6544
vnd-tie	-1.9959	0.4220	80.5438

Table 2 shows that the two VND variants achieved very similar solution quality. The transpose–insert–exchange variant (vnd-tie) obtained a slightly smaller average deviation than the transpose–exchange–insert variant (vnd-tei), which means that it was slightly better on average. It also required less total computation time. However, as shown by the statistical tests in the next subsection, this difference in solution quality is not statistically significant.

4.2 Exercise 1.1: Statistical Comparisons

A series of tests were performed to compare the algorithms, to answer the questions of : is there a statistically significant difference between the solution quality generated by the different algorithms? The t-test and the wilcoxon test are used to compare the results of the two algorithms on the same instances, it works as follows: The null hypothesis H_0 states that there is no significant difference in solution quality between the two algorithms, while the alternative hypothesis H_1 states that there is a significant difference. The test calculates a p-value, which represents the probability of observing the data if the null hypothesis were true. If the p-value is less than a chosen significance level (here we chose 0.05 like in the lecture example), we reject the null hypothesis in favor of the alternative. We use both tests because they rely on different assumptions. The paired t-test is a parametric test and focuses on differences in the mean, whereas the Wilcoxon signed-rank test is non-parametric and focuses on the median of the paired differences. The lecture example showed that the two tests may lead to different conclusions

on the same data, which is why reporting both provides a more complete statistical analysis.

Table 3: Selected statistical comparisons for Exercise 1.1.

Comparison	Paired t-test p-value	Wilcoxon p-value
best exchange random vs best insert random	4.82×10^{-33}	1.93×10^{-14}
best exchange CW vs best insert CW	5.77×10^{-36}	1.72×10^{-14}
first insert random vs best insert random	2.42×10^{-5}	3.49×10^{-5}
first insert random vs first insert CW	0.9400	0.9088
best transpose random vs best transpose CW	1.38×10^{-53}	1.72×10^{-14}

Table 3 highlights several important results. First, the comparisons between best-exchange and best-insert are highly significant both with random initialization and with CW initialization. Since the insert variants achieved smaller average deviations, these results confirm that the insert neighborhood clearly outperforms the exchange neighborhood in terms of solution quality.

Second, the comparison between first-insert random and best-insert random is also statistically significant. In this case, the first-improvement version achieved a smaller average deviation than the best-improvement version, which shows that best-improvement is not necessarily better in practice on these benchmark instances.

Third, the comparison between first-insert random and first-insert CW is not significant. This suggests that when the insert neighborhood is used, the initialization method has little influence on the final solution quality.

Finally, the comparison between best-transpose random and best-transpose CW is highly significant. Since the CW version is much closer to the best known solutions, this indicates that initialization is particularly important when the neighborhood is weak, as is the case for transpose.

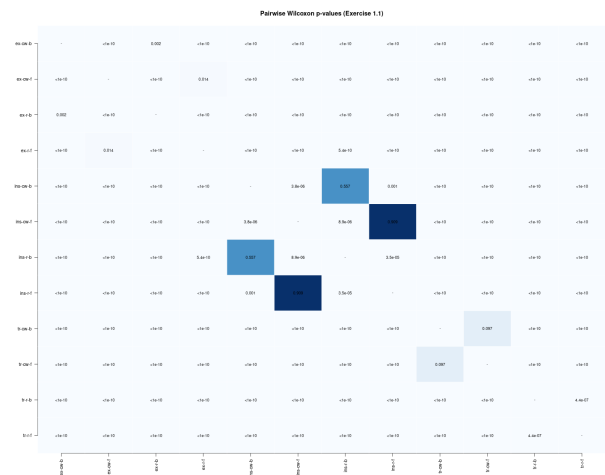


Figure 1: Pairwise Wilcoxon p-values for the 12 iterative improvement algorithms of Exercise 1.1. Smaller p-values indicate stronger evidence of a statistically significant difference in solution quality between the corresponding pair of algorithms.

Figure 1 provides a more global view of the pairwise Wilcoxon comparisons. It confirms that many algorithm pairs differ significantly, especially when comparing insert-based variants with exchange-based ones, while only a few pairs show no statistically significant difference.

4.3 Exercise 1.2: Statistical Comparison

The two VND variants were also compared using paired Student t-tests and Wilcoxon signed-rank tests.

- Average deviation vnd-tei: -2.0633
- Average deviation vnd-tie: -1.9959
- Std deviation vnd-tei: 0.4191
- Std deviation vnd-tie: 0.4220
- Paired t-test p-value: 0.2136
- Wilcoxon p-value: 0.4169

The transpose–insert–exchange variant obtained a slightly better average deviation than the transpose–exchange–insert variant, which means that it was slightly better on average. However, both statistical tests returned p-values greater than 0.05 . Therefore, no statistically significant difference in solution quality was observed between the two VND algorithms. In other words, although vnd-tie appears slightly better empirically, this difference is not strong enough to conclude that it systematically outperforms vnd-tei.

5 CONCLUSIONS

In this work, iterative improvement and variable neighborhood descent algorithms were implemented for the linear ordering problem. For Exercise 1.1, twelve iterative improvement variants were

evaluated by combining two pivoting rules, three neighborhoods, and two initialization methods. For Exercise 1.2, two VND variants were implemented using the required neighborhood orders and CW initialization.

The experimental results show that the choice of neighborhood has the strongest influence on solution quality. In particular, the insert neighborhood clearly outperformed the exchange neighborhood, while transpose was by far the weakest neighborhood. The results also showed that the impact of initialization depends on the strength of the neighborhood: for transpose, CW initialization was much better than random initialization, whereas for insert the difference between random and CW was not statistically significant. In addition, the insert neighborhood with first-improvement achieved better results than with best-improvement on the tested benchmark instances.

For the VND algorithms, the transpose–insert–exchange variant obtained a slightly better average deviation and a lower total runtime than the transpose–exchange–insert variant. However, the statistical tests showed that this difference in solution quality was not significant.

Overall, these experiments suggest that using a strong neighborhood such as insert is more important than using a more expensive pivoting rule, and that good initialization is especially important when the neighborhood is weak. Among the tested methods, insert-based algorithms provided the best balance between solution quality and computational effort.

REFERENCES